

Final Project VAE Anomaly Detection

Tristan Jones CSCI 8110

1) Introduction

Anomaly detection is a critical problem across many domains, including quality-control inspection, medical imaging, cybersecurity, and fraud detection. Because anomalous examples are often rare, expensive to label, or impossible to enumerate exhaustively, anomaly detection is commonly framed as a semi-supervised or unsupervised problem in which a model learns a compact representation of normal behavior and then identifies deviations from this learned structure as potential outliers. Reconstruction-based approaches, such as autoencoders, have been widely used for this purpose: the model is trained only on normal data and is expected to reconstruct those inputs well while producing noticeably higher reconstruction errors on anomalous inputs that fall off the learned manifold.

Variational Autoencoders (VAEs) extend classical autoencoders by imposing a probabilistic structure on the latent space, encouraging the encoder to map inputs to a smooth Gaussian distribution rather than to deterministic latent codes [1]. Instead of outputting a single latent vector, the encoder predicts the mean and variance of a multivariate Gaussian, and a latent sample is drawn using the reparameterization trick. A Kullback-Leibler divergence term regularizes the latent space, preventing overly sharp memorization and encouraging a continuous, semantically meaningful manifold [1]. These properties make VAEs attractive for anomaly detection because inputs that deviate from standard patterns typically fall outside this latent distribution and yield poor reconstructions [2].

In this project, I apply a convolutional VAE to the Fashion-MNIST dataset, a collection of 28×28 grayscale images spanning ten classes of clothing items [3]. Unlike class-based anomaly detection, in which one category is treated as usual and the others as anomalous, this work examines structural anomalies induced by corrupting otherwise normal images. Specifically, synthetic anomalies are generated using occlusions, additive Gaussian noise, and large rotations to produce inputs that deviate from the distribution of clean images in controlled ways. The VAE is trained exclusively on clean pictures from all classes and evaluated by comparing reconstruction errors between clean and corrupted samples. Anomaly detection performance is measured using AUROC, accuracy, and additional classification metrics at a fixed rate.

This project investigates three primary research questions:

1. How effectively can a VAE trained on Fashion-MNIST detect synthetic anomalies using reconstruction error?
2. How does anomaly-detection performance change as corruption severity increases?
3. Does the latent-space regularization of a VAE provide an advantage over a similarly structured deterministic autoencoder?

To address these questions, I conduct a series of experiments that analyze reconstruction-based anomaly detection across corruption types, severities, model architectures, and training durations. Performance is evaluated across a balanced mixture of typical and corrupted examples, followed by sensitivity analyses that vary noise level, occlusion size, and rotation magnitude. A matched autoencoder baseline is included to isolate the effect of the VAE's probabilistic latent modeling. Finally, I study how detection performance evolves as the VAE is trained for 10, 30, and 60 epochs. Although not intended to achieve state-of-the-art performance, this work provides a focused case study on the behavior, strengths, and limitations of VAE-based anomaly detection on a realistic but straightforward image dataset. The resulting insights help clarify when reconstruction-based methods perform well, where they break down, and how model structure and corruption characteristics influence anomaly separability.

2) Related Work

Autoencoders are widely used for unsupervised anomaly detection across modalities such as images, time series, and tabular data [1], [2]. A standard autoencoder consists of an encoder-decoder pair that compresses inputs into a low-dimensional representation and then reconstructs them. When trained only on normal data, reconstruction error functions as an anomaly score: samples that deviate from the learned manifold are expected to reconstruct poorly. This strategy has been used in industrial inspection, surveillance, and medical imaging, where anomalies are rare or difficult to annotate [3], [4]. However, classical autoencoders lack an explicit probabilistic model of the latent space, and sufficiently expressive architectures may inadvertently reconstruct off-manifold inputs with low error, reducing separation between normal and abnormal samples [5].

Variational Autoencoders (VAEs) address this limitation by introducing a probabilistic latent representation. Rather than mapping each input to a single deterministic code, the encoder outputs the parameters of a Gaussian distribution from which latent vectors are sampled [6]. Training minimizes both reconstruction loss and a Kullback-Leibler divergence term that regularizes the latent distribution toward a simple prior, typically an isotropic Gaussian. This regularization encourages smoother, more structured latent spaces and reduces memorization of training examples. In anomaly detection, VAEs often exhibit increased sensitivity to off-manifold inputs because anomalous samples tend to produce higher reconstruction errors or atypical latent embeddings [7].

Benchmark datasets frequently used in anomaly detection research include MNIST, Fashion-MNIST [3], CIFAR-10, and MVTec AD [7]. Two primary evaluation paradigms appear in the literature: class-based anomaly detection, in which a single class is designated as normal and all others treated as anomalous; and corruption-based anomaly detection, in which anomalies are generated by applying controlled perturbations such as noise injection, occlusion, or rotation [8]. The corruption-based paradigm is especially relevant for practical applications involving sensor interference, imaging artifacts, or partial object degradation. Fashion-MNIST,

with greater intra-class variability than MNIST while retaining low computational cost, serves as a convenient benchmark for reconstruction-based anomaly detection frameworks [3].

This project adopts the corruption-based evaluation paradigm to examine how a convolutional VAE compares with a structurally similar deterministic autoencoder on the Fashion-MNIST dataset. Rather than proposing a new architecture, the study focuses on how latent-space regularization, corruption type, corruption severity, and training duration influence anomaly detection performance. By combining quantitative metrics, reconstruction visualizations, and controlled sensitivity analyses, this work contributes a focused empirical investigation of VAE-based anomaly detection. This prior literature motivates the structured comparison performed here, which isolates the role of latent regularization under controlled corruption-based anomaly conditions.

3) Methodology

The VAE in this project uses a convolutional encoder-decoder architecture, which is well-suited for image data because convolutional layers efficiently capture local spatial features and generalize better to shifted or partially corrupted inputs than fully connected layers. A latent dimension of 16 was chosen to balance compression and reconstruction quality, as overly small latent spaces harm normal reconstructions while overly large ones can reduce anomaly sensitivity through memorization. Binary cross-entropy reconstruction loss was used instead of mean squared error because Fashion-MNIST images are normalized grayscale data and can be interpreted probabilistically, leading to sharper reconstructions and better gradient behavior in VAEs. A batch size of 128 provides a practical trade-off between computational efficiency and stable training, particularly for the stochastic latent sampling and KL regularization terms.

3.1 Dataset and preprocessing

```
# =====
# 1) Load Fashion-MNIST
# =====
# Dataset: 60,000 training + 10,000 test images (28x28 grayscale)
# 10 classes: T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot
print("\nLOADING FASHION-MNIST DATASET")
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
print(f"✓ Dataset loaded successfully")
print(f" Train data shape: {x_train.shape}")
print(f" Test data shape: {x_test.shape}")

# Normalize and add channel dimension
print("\nPREPROCESSING DATA")
x_train = x_train.astype("float32") / 255.0 # Convert to [0, 1] range
x_test = x_test.astype("float32") / 255.0
x_train = np.expand_dims(x_train, -1) # (N, 28, 28, 1) for Conv2D
x_test = np.expand_dims(x_test, -1)
print(f"✓ Normalized pixel values to [0, 1]")
print(f"✓ Added channel dimension")
print(f" Train shape: {x_train.shape}")
print(f" Test shape: {x_test.shape}")
```

From line 371, the Fashion-MNIST dataset was loaded using Keras and normalized into the [0,1] range. Here, all 10 classes are treated as normal data. No Data Augmentation or corruption is applied at all during the training phase.

3.2 Corruption/anomaly generation

Because the dataset contains no labeled anomalies, synthetic corruptions are applied to clean images at test time to generate controlled anomalous samples. Three corruption types are used to reflect common real-world imaging issues: occlusion via black-square masks to simulate missing data or sensor obstructions, additive Gaussian noise to model degraded or noisy acquisition conditions with varying severity, and rotation transformations to introduce geometric misalignment while preserving pixel intensities. In addition, a combined corruption setting applies both occlusion and noise to create more challenging anomalies. Together, these perturbations span a range of deviations from the learned manifold of normal clothing images, enabling a systematic evaluation of anomaly detection sensitivity.

Black box occlusion (add_black_box)

```
def add_black_box(x, box_size=8):
    x_corrupt = x.copy()
    N, H, W, C = x.shape

    for i in range(N):
        img = x[i, :, :, 0]
        content_mask = img > 0.1

        if np.sum(content_mask) > box_size * box_size:
            valid_positions = []
            for top in range(H - box_size):
                for left in range(W - box_size):
                    box_region = content_mask[top:top+box_size, left:left+box_size]
                    if np.sum(box_region) > (box_size * box_size * 0.3):
                        valid_positions.append((top, left))

            if valid_positions:
                top, left = valid_positions[np.random.randint(len(valid_positions))]
            else:
                top = np.random.randint(0, H - box_size)
                left = np.random.randint(0, W - box_size)
        else:
            top = np.random.randint(0, H - box_size)
            left = np.random.randint(0, W - box_size)

        x_corrupt[i, top:top+box_size, left:left+box_size, :] = 0.0

    return x_corrupt
```

From line 39, essentially, this adds a black box over the image/object to simulate the occlusion caused by a broken sensor. In this code, it looks at the image and ensures the box sits on top of the actual object. Earlier versions placed occlusions randomly, including background-only regions; the updated method ensures that occlusions intersect salient object regions

Gaussian noise (add_gaussian_noise)

```
def add_gaussian_noise(x, sigma=0.3):  
    noise = np.random.normal(0.0, sigma, size=x.shape)  
    x_noisy = np.clip(x + noise, 0.0, 1.0)  
    return x_noisy
```

From line 90 onward, Gaussian noise is added, which appears as TV static in the image, making it harder to determine where the object might actually be.

Rotation (rotate_images, apply_rotation_only)

```
def rotate_images(x, max_angle=1.2):  
    x_rot = x.copy()  
    N = x.shape[0]  
  
    for i in range(N):  
        angle = np.random.uniform(-max_angle, max_angle)  
        angle_degrees = angle * 180 / np.pi  
  
        rotation_layer = tf.keras.layers.RandomRotation(  
            factor=(angle_degrees / 360.0, angle_degrees / 360.0),  
            fill_mode="constant",  
            fill_value=0.0  
        )  
  
        x_rot[i:i+1] = rotation_layer(x[i:i+1], training=True).numpy()  
  
    return x_rot
```

```
def apply_rotation_only(x, max_angle=1.2):  
    return rotate_images(x, max_angle=max_angle)
```

From line 95 onward, this rotates the image by a random amount each time.

Combined corruption (make_corrupted_set)

```
def make_corrupted_set(x_clean):  
    x_occ = add_black_box(x_clean, box_size=8)  
    x_noisy = add_gaussian_noise(x_occ, sigma=0.3)  
    return x_noisy
```

From line 122, this adds both the occlusion and the Gaussian noise simultaneously.

3.3 VAE architecture

The Variational Autoencoder used in this project follows a standard convolutional encoder-decoder structure with a 16-dimensional latent space. The model consists of four main components: the encoder, the reparameterization module, the decoder, and a custom training loop required to compute the VAE loss.

Encoder

```
encoder_inputs = layers.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, 3, strides=2, padding="same", activation="relu")(encoder_inputs)
x = layers.Conv2D(64, 3, strides=2, padding="same", activation="relu")(x)
x = layers.Flatten()(x)
x = layers.Dense(128, activation="relu")(x)

z_mean = layers.Dense(latent_dim, name="z_mean")(x)
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)
```

From line 457, the encoder maps a $28 \times 28 \times 1$ input image to a compact latent representation using two strided convolutional layers with 32 and 64 filters, respectively, to progressively downsample and extract features. The resulting feature maps are flattened and passed through a 128-unit dense layer, after which two separate dense layers output the latent mean and log-variance vectors with a dimensionality of 16, parameterizing the VAE's latent distribution.

Reparameterization

```
def sampling(args):
    z_mean, z_log_var = args
    epsilon = tf.random.normal(shape=tf.shape(z_mean))
    return z_mean + tf.exp(0.5 * z_log_var) * epsilon

z = layers.Lambda(sampling, name="z")([z_mean, z_log_var])
```

From line 479, the sampling function implements the reparameterization trick, where a latent vector is generated as the sum of the latent mean and a scaled noise term drawn from a standard normal distribution. This formulation separates the stochastic component from the learned parameters, allowing the model to express randomness while keeping the computation differentiable. As a result, gradients can be propagated through the sampling operation during training, enabling end-to-end backpropagation despite the presence of stochastic latent variables.

Decoder

```
latent_inputs = layers.Input(shape=(latent_dim,))
x = layers.Dense(7 * 7 * 64, activation="relu")(latent_inputs)
x = layers.Reshape((7, 7, 64))(x)
x = layers.Conv2DTranspose(64, 3, strides=2, padding="same", activation="relu")(x)
x = layers.Conv2DTranspose(32, 3, strides=2, padding="same", activation="relu")(x)
decoder_outputs = layers.Conv2DTranspose(1, 3, activation="sigmoid", padding="same")(x)
decoder = models.Model(latent_inputs, decoder_outputs, name="decoder")
```

From line 492, the decoder reconstructs an image from the latent vector by first expanding it with a dense layer into a 7×7×64 feature tensor, then reshaping it back into spatial form. Two transposed convolution layers with 64 and 32 filters progressively upsample the feature maps back to the original 28×28 resolution while learning how to rebuild image structure. A final transposed convolution with one output channel and a sigmoid activation produces the reconstructed 28×28×1 image, with pixel values constrained to the 0-1 range.

Custom training step and VAE Loss

```
class VAE(models.Model):
    def __init__(self, encoder, decoder, beta=1.0, **kwargs):
        super(VAE, self).__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.beta = beta

    def train_step(self, data):
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)
            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(tf.keras.losses.binary_crossentropy(data, reconstruction), axis=(1, 2))
            )
            kl_loss = -0.5 * tf.reduce_mean(
                tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var), axis=1)
            )
            total_loss = reconstruction_loss + self.beta * kl_loss
        grads = tape.gradient(total_loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
        return {"loss": total_loss}
```

From line 509, this implementation uses a custom `train_step()` method because a Variational Autoencoder cannot rely on Keras's standard single-loss training loop. A VAE must compute both reconstruction loss and KL divergence inside the training procedure, and gradients must propagate through the stochastic sampling step. Defining the loss manually within `train_step()` ensures correct gradient flow, proper weighting of the KL term, and end-to-end optimization of the encoder, decoder, and sampling mechanism.

3.4 Baseline Autoencoder

```
def build_vanilla_autoencoder(latent_dim=16):
    encoder_inputs = layers.Input(shape=(28, 28, 1))
    x = layers.Conv2D(32, 3, strides=2, padding="same", activation="relu")(encoder_inputs)
    x = layers.Conv2D(64, 3, strides=2, padding="same", activation="relu")(x)
    x = layers.Flatten()(x)
    x = layers.Dense(128, activation="relu")(x)
    z = layers.Dense(latent_dim, activation="relu", name="latent")(x)

    encoder = models.Model(encoder_inputs, z, name="ae_encoder")

    latent_inputs = layers.Input(shape=(latent_dim,))
    x = layers.Dense(7 * 7 * 64, activation="relu")(latent_inputs)
    x = layers.Reshape((7, 7, 64))(x)
    x = layers.Conv2DTranspose(64, 3, strides=2, padding="same", activation="relu")(x)
    x = layers.Conv2DTranspose(32, 3, strides=2, padding="same", activation="relu")(x)
    decoder_outputs = layers.Conv2DTranspose(1, 3, activation="sigmoid", padding="same")(x)

    decoder = models.Model(latent_inputs, decoder_outputs, name="ae_decoder")

    ae_outputs = decoder(z)
    autoencoder = models.Model(encoder_inputs, ae_outputs, name="autoencoder")

    return autoencoder
```

From line 426, the `build_vanilla_autoencoder` function implements a standard convolutional autoencoder using the same encoder and decoder architecture as the VAE, but with a deterministic latent representation and no KL divergence term. The model is trained solely to minimize mean squared error between the input images and their reconstructions. This design provides a fair baseline comparison, allowing the effect of VAE latent regularization to be isolated and evaluated in terms of its impact on anomaly detection performance.

3.5 Anomaly score and evaluation metrics

Anomaly detection in this project is based on the idea that a VAE trained only on clean images should reconstruct regular inputs well but should struggle when presented with corrupted or out-of-distribution images. The difference between the input image and its reconstruction provides a quantitative signal of how abnormal the input appears to the model.

Anomaly score (reconstruction error).

```
def reconstruction_errors(model, x):  
    recon = model.predict(x, batch_size=256, verbose=0)  
    errors = np.mean((x - recon) ** 2, axis=(1, 2, 3))  
    return errors
```

From line 681, this study computes the mean squared error (MSE) between the input and its reconstruction for each image. This value is averaged over all pixels. Higher reconstruction error indicates that the model was unable to faithfully reproduce the image, which is treated as evidence of an anomaly. Clean (normal) test images are assigned the label 0, and corrupted images are assigned the label 1. The reconstruction error serves as the anomaly score used for binary evaluation.

Threshold-free metric: AUROC.

```
def plot_vae_vs_ae_comparison(vae_results, ae_results):  
    corruption_types = list(vae_results.keys())  
    vae_aurocs = [vae_results[c] for c in corruption_types]  
    ae_aurocs = [ae_results[c] for c in corruption_types]  
  
    x = np.arange(len(corruption_types))  
    width = 0.35  
  
    fig, ax = plt.subplots()  
    ax.bar(x - width/2, vae_aurocs, width)  
    ax.bar(x + width/2, ae_aurocs, width)  
  
    ax.set_xticks(x)  
    ax.set_xticklabels(corruption_types)  
  
    plt.tight_layout()  
    plt.savefig('vae_visualizations/vae_vs_ae_comparison.png', dpi=150)  
    plt.close()
```

Simplified function from line 301 to evaluate how well reconstruction error separates normal and corrupted images across all possible thresholds, I compute the Area Under the Receiver Operating Characteristic Curve (AUROC). AUROC reflects the model's ability to consistently assign higher scores to anomalies than to standard samples. A score of 0.5 corresponds to random guessing, while a score of 1.0 indicates perfect discrimination.

Fixed operating point (95th percentile threshold).

```
threshold = np.percentile(err_normal, 95)
y_pred = (scores > threshold).astype(int)
```

From line 711 onward, performance is evaluated at a single detection threshold. The threshold is set to the 95th percentile of reconstruction errors among clean test images, meaning that approximately 5% of standard samples are allowed to be false positives. Any image whose reconstruction error exceeds this threshold is classified as anomalous.

Classification metrics.

```
precision = precision_score(labels, y_pred)
recall = recall_score(labels, y_pred)
f1 = f1_score(labels, y_pred)
cm = confusion_matrix(labels, y_pred)
```

From line 720, I compute several evaluation metrics using the predicted labels at this threshold to assess anomaly detection performance. Accuracy reflects the overall proportion of correctly classified samples, while precision measures the percentage of predicted anomalies that are truly anomalous. Recall quantifies the number of actual anomalies the model successfully detects, and the F1-score provides a balanced measure by taking the harmonic mean of precision and recall. In addition, a confusion matrix summarizes the counts of true negatives, false positives, false negatives, and true positives. Together, these metrics offer a comprehensive view of the model's performance, capturing both its threshold-free separability as measured by AUROC and its practical classification behavior at a fixed operating point.

4) Experiments and Results

This section presents the empirical evaluation of the VAE-based anomaly detection system. All experiments were conducted on the Fashion-MNIST test set, treating clean test images as standard samples and corrupted versions as anomalies. Performance is assessed using reconstruction error, threshold-based classification metrics, and AUROC. Additional analyses investigate the severity and type of corruption, compare with a plain autoencoder, and determine sensitivity to training duration.

4.1 Main Anomaly Detection Performance (Combined Corruption)

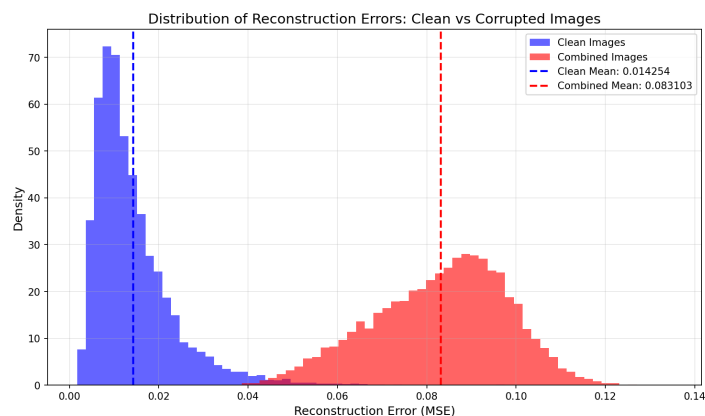


Figure 1: Distribution of reconstruction errors for clean vs. combined-corrupted images.

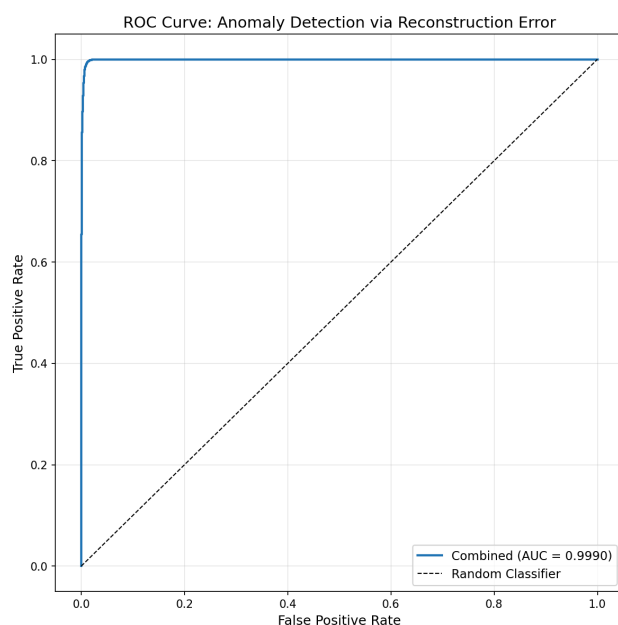


Figure 2: ROC curve for combined-corruption anomaly detection.

Evaluation was performed on 10,000 clean test images and 10,000 corrupted counterparts. The primary assessment considers the combined-corruption setting, in which black-box occlusions and Gaussian noise are applied simultaneously to test pictures. As shown in Figure 1, reconstruction errors for clean and corrupted images form two well-separated distributions. Clean samples cluster tightly around low error values, reflecting the VAE’s ability to reconstruct regular inputs with high fidelity. In contrast, combined-corruption anomalies exhibit substantially higher reconstruction errors, shifting the distribution markedly to the right with minimal overlap between the two groups.

Using reconstruction error as the anomaly score, the model achieves an AUROC of 0.9990, indicating near-perfect separability between standard and corrupted inputs (Figure 2).

Applying a fixed detection threshold at the 95th percentile of clean-image reconstruction errors yields high accuracy, precision, recall, and F1-score, with very few false positives or false negatives. Although the resulting performance appears exceptionally strong, it is consistent with expectations in a reconstruction-based anomaly detection setting: the VAE is trained exclusively on clean images and is therefore highly sensitive to distortions that drive inputs off the learned manifold. Strong corruptions, such as occlusions and noise, cause reconstruction failures that translate into robust anomaly scores, which explains the excellent detection performance observed here.

4.2 Qualitative Reconstruction Examples

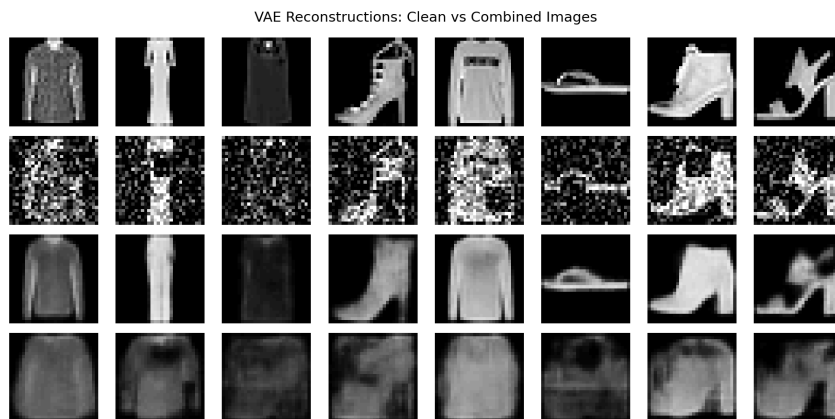


Figure 3: Clean vs. combined-corruption reconstructions.

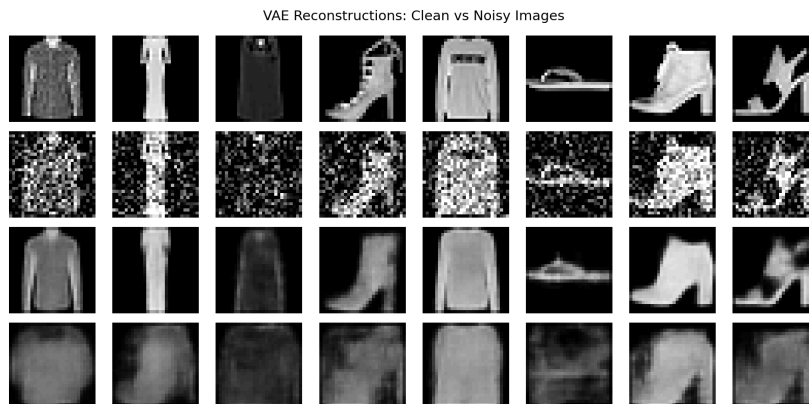


Figure 4: Clean vs. noise-corrupted reconstructions.

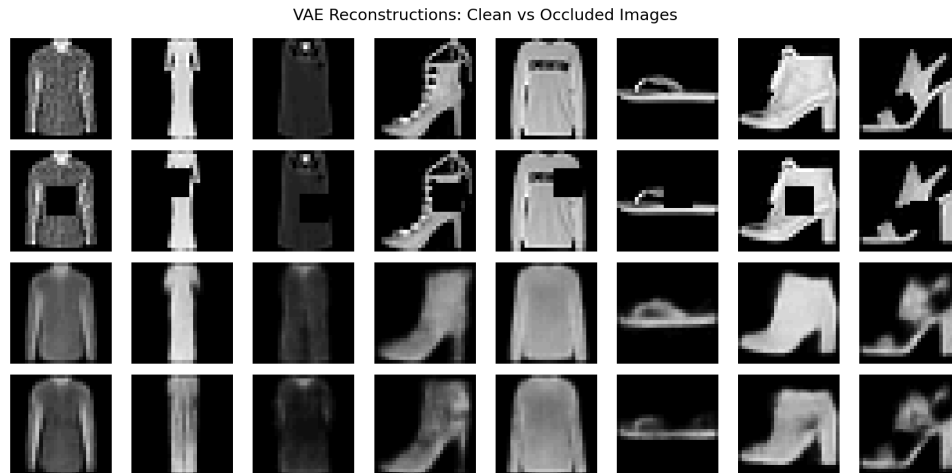


Figure 5: Clean vs. occluded reconstructions.

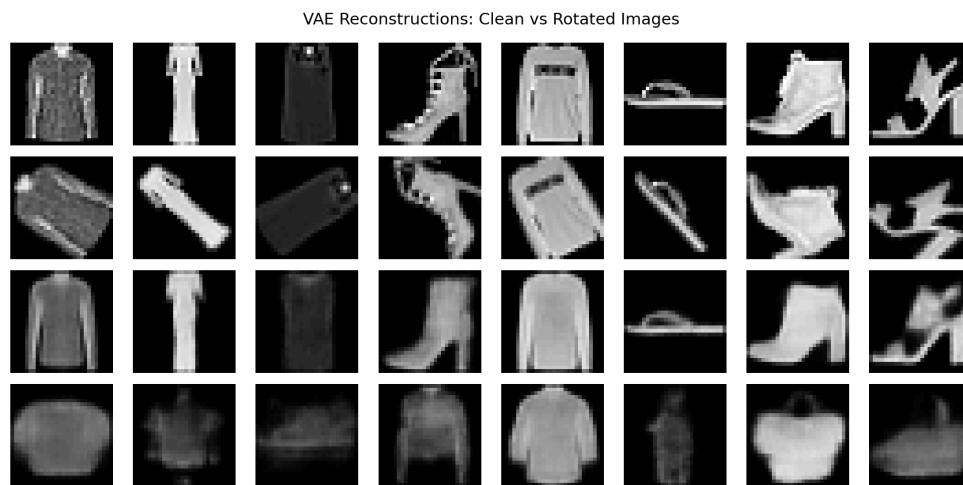


Figure 6: Clean vs. rotated reconstructions.

The VAE reconstruction visualizations are generated by the `plot_side_by_side_reconstructions()` function, which produces a four-row comparison grid for a given corruption type. The function first takes a set of clean sample images and applies the selected corruption, occlusion, noise, rotation, or a combination of these. Both the clean and corrupted versions of these images are then passed through the trained VAE using `vae_model.predict()` to obtain their reconstructions. The resulting figures display four rows: the original clean images in the first row, the corrupted “anomalous” inputs in the second row, the VAE reconstructions of the clean images in the third row, and the VAE reconstructions of the corrupted pictures in the fourth row. Each column corresponds to one image sample, allowing direct side-by-side comparison of how corruption affects reconstruction quality. These visualizations illustrate how the VAE accurately reconstructs regular inputs but produces noticeably distorted outputs when presented with anomalous images.

Qualitative results provide further insight into how the VAE handles different types of corruption. Figures 3-6 show 4-row grids for clean, corrupted, clean-reconstructed, and corrupted-reconstructed images. Across all corruption types, the VAE reconstructs clean garments with reasonable structure and smoothness, despite some blurring due to the low spatial resolution and the regularization imposed by the KL term. In contrast, reconstructions of corrupted images tend to “snap back” toward plausible clothing shapes rather than reproducing the corruptions themselves. Occlusions, noise patches, and severe rotations are not retained in the output; instead, generally, the decoder generates approximate garment silhouettes consistent with the learned manifold. In other words, because it is trained only on clean images, it tries to reconstruct the learned clean version of the image.

4.3 Effect of Corruption Type

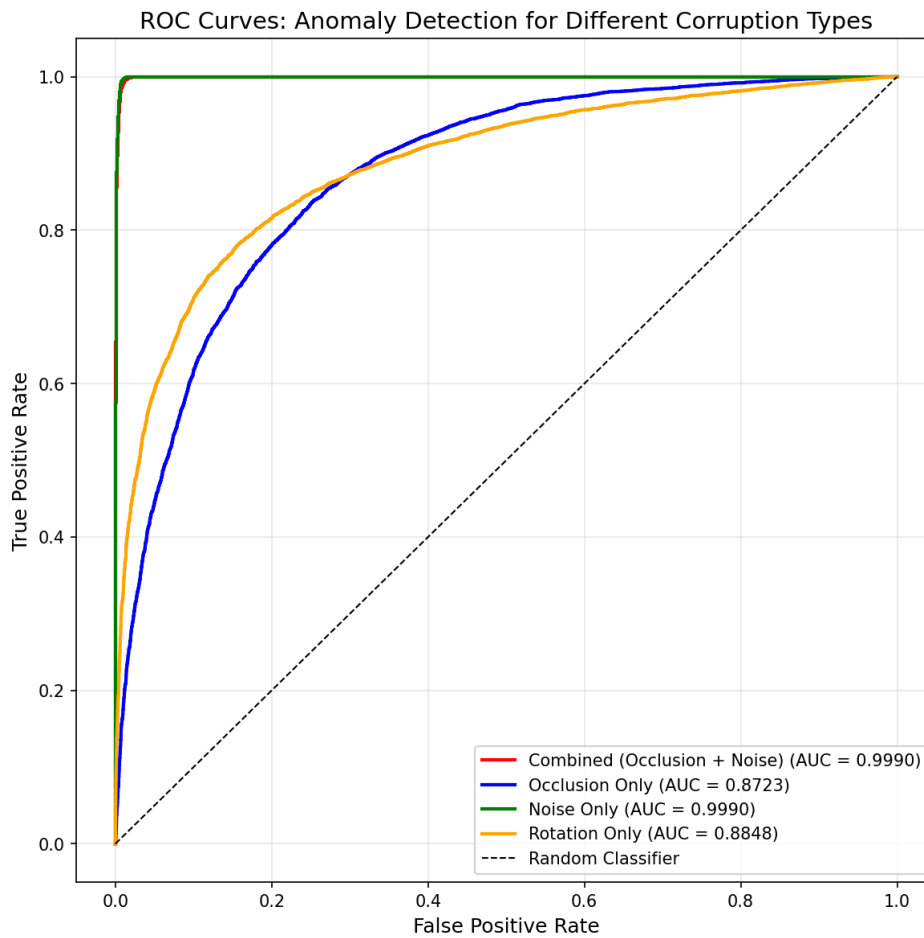


Figure 7: ROC curves for different corruption types.

To compare sensitivity to specific anomaly types, separate anomaly sets were created for occlusion-only, noise-only, rotation-only, and combined corruption. The resulting ROC curves appear in Figure 7. Noise-only and combined corruption are detected extremely well, both achieving AUROC values close to 1.0. Rotation-only and occlusion-only corruptions are slightly

more challenging, achieving AUROC values of approximately 0.88 for rotation and 0.87 for occlusion in this experiment. These results indicate that photometric distortions, such as noise, perturb the reconstruction more strongly than geometric distortions, which the decoder sometimes partially normalizes back into plausible clothing structures. Overall, the VAE consistently outperforms random guessing (AUROC = 0.5) by a wide margin for all corruption types, with the strongest performance occurring when corruptions significantly alter pixel intensities.

4.4 Sensitivity to Corruption Severity

Noise Severity

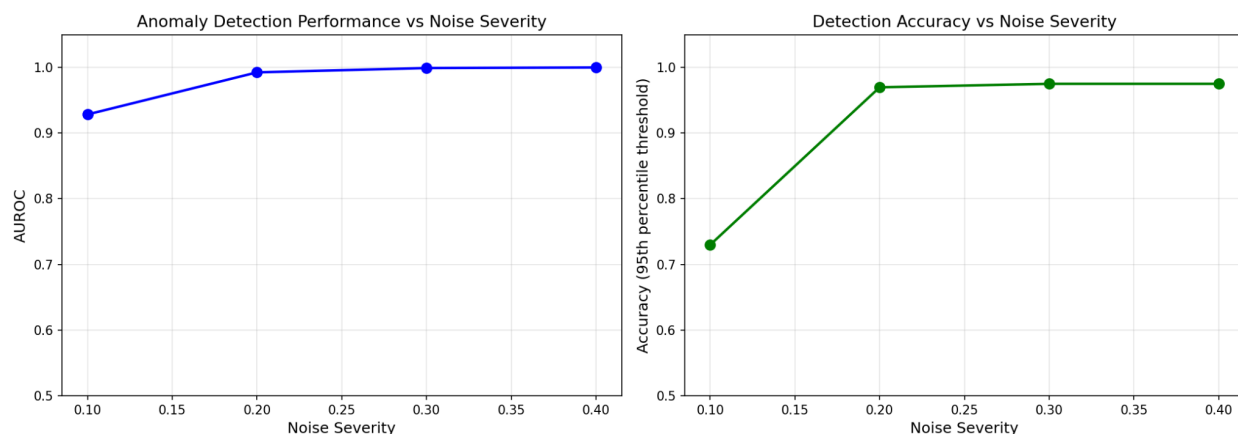


Figure 8: Sensitivity to noise severity.

Noise standard deviation was varied from $\sigma = 0.1$ to 0.4 to analyze the effect of corruption magnitude on detection performance. As shown in Figure 8, both AUROC and threshold-based accuracy increase monotonically with noise severity. Mild noise introduces subtle photometric deviations that the VAE may partially reconstruct. Still, more substantial noise perturbations move inputs farther from the manifold of clean training images, producing larger reconstruction errors. This trend reflects a general principle of reconstruction-based anomaly detection: anomalies become easier to detect as their distance from the normal-data manifold increases.

Occlusion Size

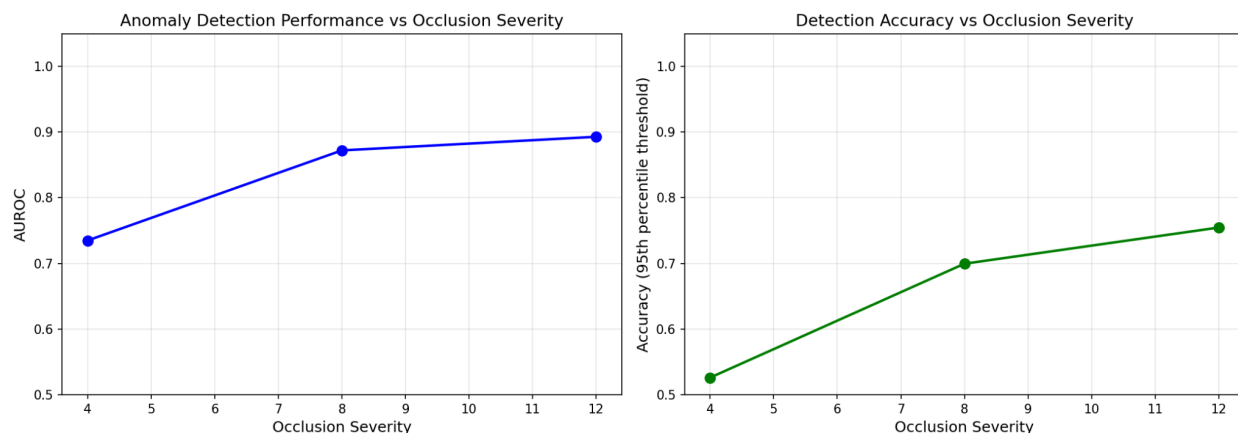


Figure 9: Sensitivity to occlusion severity.

Occlusion severity was varied by adjusting the size of the black box (4, 8, and 12 pixels). Figure 9 shows that both AUROC and accuracy increase with occlusion size. Small occlusions sometimes lie in low-information regions (e.g., black background), making them harder to detect. Larger occlusions disrupt garment structure more visibly, yielding higher reconstruction errors and better separability. As such, we get better results as we increase the box size.

Rotation Angle

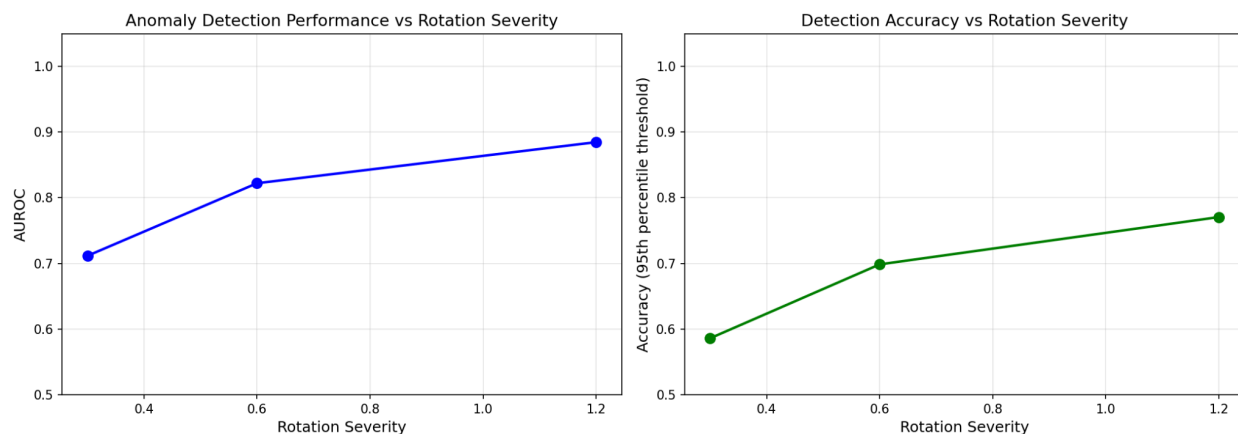


Figure 10: Sensitivity to rotation angle.

Rotation severity was evaluated at maximum rotation angles of approximately 17° , 34° , and 69° . AUROC and accuracy both increase with rotation magnitude, although geometric distortions generally remain more challenging than noise or occlusions. The decoder often maps moderately rotated objects back toward upright shapes, reducing anomaly scores at smaller angles. Larger rotations deviate further from the manifold, producing clearer reconstruction inconsistencies. Overall, sensitivity to rotation severity provides insight into the VAE's reliance on canonical spatial structure in its learned representation.

4.5 Comparison with a Plain Autoencoder

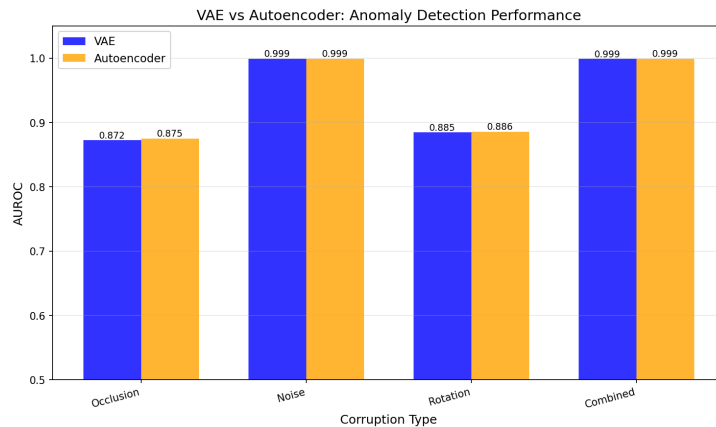


Figure 11: AUROC comparison between VAE and plain autoencoder.

To evaluate the impact of latent regularization, a convolutional autoencoder (AE) with an architecture comparable to the VAE but without the KL divergence term was trained under identical conditions. AUROC results for both models across all corruption types are shown in Figure 11. Contrary to the expectation that the VAE’s structured latent space would yield superior anomaly detection, the two models perform nearly identically in this setting. For noise and combined corruption, both models achieve near-perfect AUROC values; for occlusion and rotation, performance differences are within approximately 0.001, with the AE slightly outperforming the VAE in some cases.

These findings suggest that the simplicity of Fashion-MNIST and the strong, apparent corruptions used here make the anomaly detection task relatively easy. Both models can accurately reconstruct clean images but fail to reconstruct heavily corrupted images, resulting in significant reconstruction-error gaps that dominate the anomaly score. In such cases, the benefits of latent regularization do not manifest strongly; however, VAEs may still offer advantages in more complex domains, subtler anomaly scenarios, or when operating under constraints that require structured latent representations.

4.6 Training Duration Study

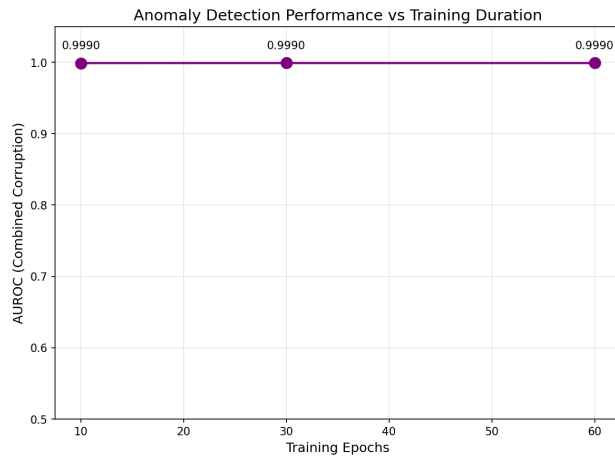


Figure 12: AUROC vs. training epochs.

To investigate the effect of training duration on anomaly detection performance, the VAE was trained for 10, 30, and 60 epochs and evaluated on the combined-corruption anomaly set. As shown in Figure 12, AUROC values remain uniformly high across all training durations, indicating that the model converges rapidly on Fashion-MNIST and that additional training epochs yield minimal observable change in detection performance.

Importantly, the near-identical AUROC values do not imply that longer training provides no benefit or that the model cannot overfit. Instead, the anomaly detection task in this study is relatively simple: anomalies are strongly out-of-distribution relative to clean training images, and even partially trained models exhibit pronounced reconstruction failures on corrupted inputs. Thus, AUROC saturates quickly and becomes insensitive to subtle reconstruction differences that might emerge with additional training. In more complex datasets or under subtle anomaly conditions, the impact of training duration might be more pronounced. In this experiment, however, performance plateaus early due to the straightforward nature of the anomalies and the robustness of reconstruction-based detection to large distribution shifts.

5) Discussion

This project demonstrated that a Variational Autoencoder trained exclusively on clean Fashion-MNIST images can reliably distinguish normal from corrupted inputs using reconstruction error as the anomaly score. Across all experiments, the VAE achieved near-perfect AUROC in the combined-corruption setting and maintained high accuracy at a fixed 95th-percentile threshold. These results arise because corrupted inputs consistently produce substantially higher reconstruction errors than clean samples, enabling clear separability. Detection difficulty varied across corruption types: strong pixel-level noise and large occlusions were easiest to detect, whereas mild geometric distortions, such as slight rotations, were more challenging to detect. Sensitivity analyses further showed that anomaly separability increases predictably with corruption severity, reflecting greater deviation from the learned manifold of normal clothing images. Comparison with a plain autoencoder revealed that both models performed similarly in this setting. Although the VAE offers theoretical advantages through latent-space regularization, the simplicity of Fashion-MNIST and the severity of the applied corruptions limit the observable impact of these differences.

The VAE's behavior can be interpreted through its manifold-learning properties. Trained only on clean examples, the model develops a compact approximation of the latent structure of normal clothing images. When presented with corrupted or structurally inconsistent inputs, the decoder tends to reconstruct the nearest plausible clean example rather than reproducing the corruption. This generates significant reconstruction discrepancies, which drive high anomaly scores. However, this exact mechanism highlights significant limitations of reconstruction-based anomaly detection. VAEs tend to produce smooth, low-frequency reconstructions that ignore fine details; they are therefore robust to obvious anomalies but vulnerable to subtle deviations that remain close to the manifold. Slight texture variations, minor artifacts, or localized defects may be reconstructed cleanly and thus be assigned incorrect low-anomaly scores. The low spatial resolution of Fashion-MNIST further restricts the model's ability to detect minor, fine-grained defects. The qualitative reconstructions in this study illustrate this inductive bias clearly: corrupted inputs are mapped back to simplified garment shapes, beneficial here but potentially problematic in real-world anomaly detection.

Several limitations constrain the conclusions drawn from these experiments. First, all anomalies were synthetic and do not reflect the full diversity of real-world defects. Second, the Fashion-MNIST dataset is low-resolution and has limited structural complexity, which means results may not generalize to richer datasets. Third, no hyperparameter exploration was performed for latent dimensionality, KL weighting, optimization strategies, or decoder depth. Finally, anomaly detection performance was evaluated in a balanced scenario in which anomalous and normal samples were equally represented; in practice, anomalies may only represent $<1\%$ of the data. These limitations clarify both the strengths and boundaries of VAE-based reconstruction models and highlight the need for more challenging datasets and subtler anomalies to evaluate real-world usefulness.

6. Conclusion

This project investigated the use of a Variational Autoencoder for anomaly detection on the Fashion-MNIST dataset using reconstruction error as the anomaly score. Trained exclusively on clean images and evaluated on several forms of synthetic corruption, the VAE consistently distinguished normal from anomalous inputs with high accuracy and near-perfect AUROC. Increasing corruption severity led to clearer separability, highlighting how reconstruction-based detectors excel when anomalies lie far from the manifold of normal images. Comparisons with a deterministic autoencoder showed nearly identical performance, suggesting that the simplicity of Fashion-MNIST and the magnitude of the anomalies limit the practical benefits of latent-space regularization. Training-duration experiments further demonstrated rapid convergence with minimal improvement beyond roughly 30 epochs.

The project set out to answer three main research questions. First, a VAE trained solely on clean data can detect synthetic anomalies with extremely high reliability under strong corruption. Second, anomaly-detection performance improves monotonically with corruption severity. Third, the VAE's latent regularization provides only marginal benefit over a standard autoencoder in this setting. Overall, the results confirm that VAEs can serve as effective anomaly detectors when anomalies are strongly off-manifold, while also underscoring essential considerations, including dataset complexity, anomaly subtlety, and the limitations of reconstruction loss, which should guide future work toward more realistic anomaly-detection tasks.

7. References and Resources Used

- [1] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” arXiv:1312.6114, 2014.
- [2] T. Schlegl, P. Seeböck, S. M. Waldstein, U. Schmidt-Erfurth, and G. Langs, “Unsupervised Anomaly Detection with Generative Adversarial Networks to Guide Marker Discovery,” in Proc. IPMI, 2017, pp. 146-157.
- [3] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms,” arXiv:1708.07747, 2017.
- [4] I. Goodfellow et al., “Generative Adversarial Nets,” in Advances in Neural Information Processing Systems (NeurIPS), 2014.
- [5] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” arXiv:1409.1556, 2015.
- [6] L. Ruff et al., “Deep One-Class Classification,” in Proc. ICML, 2018, pp. 4393-4402.
- [7] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, “MVTec AD: A Real-World Dataset for Unsupervised Anomaly Detection,” in Proc. CVPR, 2019, pp. 9592-9600.
- [8] E. Nalisnick, A. Matsukawa, Y. Teh, D. Gorur, and B. Lakshminarayanan, “Do Deep Generative Models Know What They Don’t Know?” in Proc. ICLR, 2019.
- [9] OpenAI, *ChatGPT*, Version GPT-5.1, San Francisco, CA, USA.
- [10] GitHub, *GitHub Copilot*, Version 2025, San Francisco, CA, USA.
- [11] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” Google, 2016. Available: <https://www.tensorflow.org>
- [12] F. Chollet et al., “Keras: Deep Learning for Humans,” GitHub Repository, 2015. Available: <https://keras.io>
- [13] C. R. Harris et al., “Array programming with NumPy,” **Nature**, vol. 585, pp. 357-362, 2020.
- [14] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” **Computing in Science & Engineering**, vol. 9, no. 3, pp. 90-95, 2007.
- [15] F. Pedregosa et al., “Scikit-Learn: Machine Learning in Python,” **Journal of Machine Learning Research**, vol. 12, pp. 2825-2830, 2011.